CFD_HW_2

姓名：梁祝旸
学号：12532299
课程：计算流体力学
日期：2026-03-16

Question 5: Code-Explanation

(a) Using the lecture notes as a guide, explain what flow problem this code is trying to solve;

The code simulates the transient development of a 1-D viscous channel flow driven by a constant pressure gradient.
The governing equation is :

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2} + g$$

With a constant body force **g** :

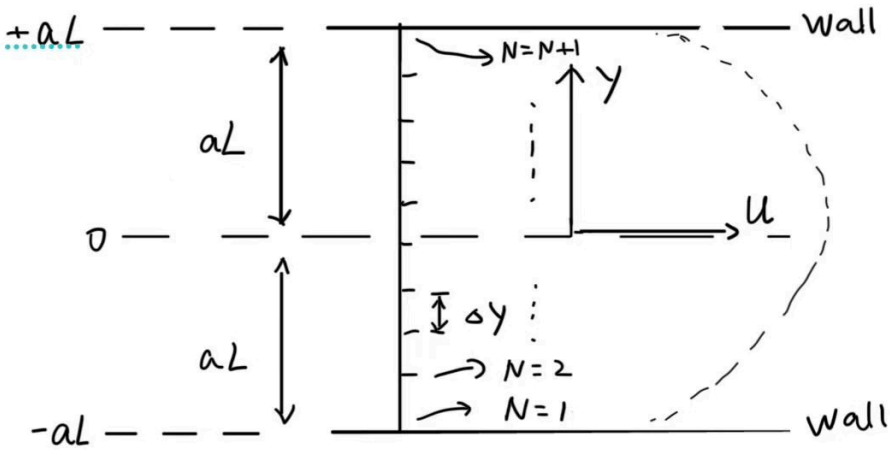
$$g = \frac{2\nu u_0}{a_L^2}$$

And B.C.s :

$$\begin{cases} u(-a_L, t) = 0 \\ u(a_L, t) = 0 \end{cases}$$

(b) Provide a sketch of the flow domain and the locations of the grid points;

The plot is kind of like :



As a 1-D flow, the grid points are just saperated along **y**-axis. And the number of points is **N + 1**. The sketch of flow shape is just a **sketch**, the **u** at walls have to be **0**.

(c) Describe, in your own words, all the input parameters for this code and their meaning;

```
integer,parameter      :: N=8, Ntime=64
real*8,parameter       ::  dt = 0.032d0

real*8,parameter       :: u0=1.0d0, aL=1.0d0, anu=0.1
```

N means the number of grids. (number of grid ponits is **N + 1**)

Ntime is used for control the frequency for print out the theory solution for comparison every **Ntime** steps.

dt is the **time step** for the explicit Euler integration.

u0 is the **maxximum velocity** in the steady case, whitch should be the steady velocity at **y = 0**.

al is the half width of the channel.

anu is the **kinematic viscosity** ν of the fluid.

(d) Describe, in your own words, what are the output data of the code?

there are two kinds of output in this code.

1. Output all the parameters:

```
write(*,*) 'anu, dt, aL, dy, CFL=', anu, dt, aL, dy, CFL
...
write(*,*) 'Tend, dt, Ntime, nsteps=', Tend, dt, Ntime, nsteps
```

Except those constant parameters described at question(C), the other parameters are definded as :

dy means the length of each unit grid.

CFL decides the stability parameter for explicit Euler integration. In the code is :

$$CFL = \frac{\nu \Delta t}{\Delta y^2}$$

In this 1-D flow, CFL should smaller than 0.5.

Tend is shown as :

$$Tend = \frac{3(aL)^2}{\nu}$$

It is 3 times the viscous diffusion time scale(in order to reach the steady state), and **Tend** decides how long the simulation is.(physical time)

And **nstep** is :

$$nstep = \frac{Tend}{dt}$$

Eventually **nstep** is the loop time steps.(Iteration time)

2. Output the calculation results and compare with the theoretical results:

```
write(55,100)ycc,theory1,u(j)/u0,abs(u(j)-theory1)/theory1
...
100     format(2x, 4f16.12)
```

This code asks computer to write the results in a file named "fort.55".

The 4 numbers are :

ycc: the coordinate of the data points.
theory1: the theoretical solution at **ycc** point.
u(j)/u0: the numerical solution at **ycc** point.(non-dim velocity)
abs(u(j)-theory1)/theory1: the error between the theoretical solution and numerical solution.

Question 6: Code-Running

(a)

As the time step size is fixed :

$$dt = \frac{0.32dy^2}{\nu} = \frac{0.32 \times 4L^2}{\nu N^2} = \frac{12.8}{N^2}$$

In order to catch the non-dimensional numbers *as accurate as possible*, I have to change **Ntime** in each simulation.
We have：

$$k = 0.2,\ 1.0,\ 5.0 = \frac{\nu t}{L^2} = \frac{\nu \mathbf{m} dt}{L^2}$$

So, we can get **m** :

$$\mathbf{m} = \frac{L^2 k}{\nu dt} = \frac{k}{0.1dt}$$

As **k** is 2\10\50 times of ν, choose **k** = 0.2 to get **Ntime** , then I can get get the velocity we want in the output file **fort.**
The results will be a large numbers of data, but I choose give this complex problem to AI in the plotting steps. It have to find the data that the output times are **1** , **5** and **25**. And for convenience, I will add a output number in each output files：

```
integer                  :: j,it,nsteps,k,output_count
...
      if(mod(it,Ntime).eq.0) then
        ! analytical solution with the same N
         output_count = output_count + 1
...
        write(32,100) REAL(output_count),ycc,theory1,u(j)/u0,abs(u(j)-theory1)/theory1
...
100     format(2x, 5f16.12)
```

I run the code 3 times and change 3 main variables **N** , **dt** and **Ntime** :

simulation	N	dt	Ntime
1	8	0.2	10
2	16	0.05	40
3	32	0.0125	160

At the same time, I have to change the **Tend** to be :

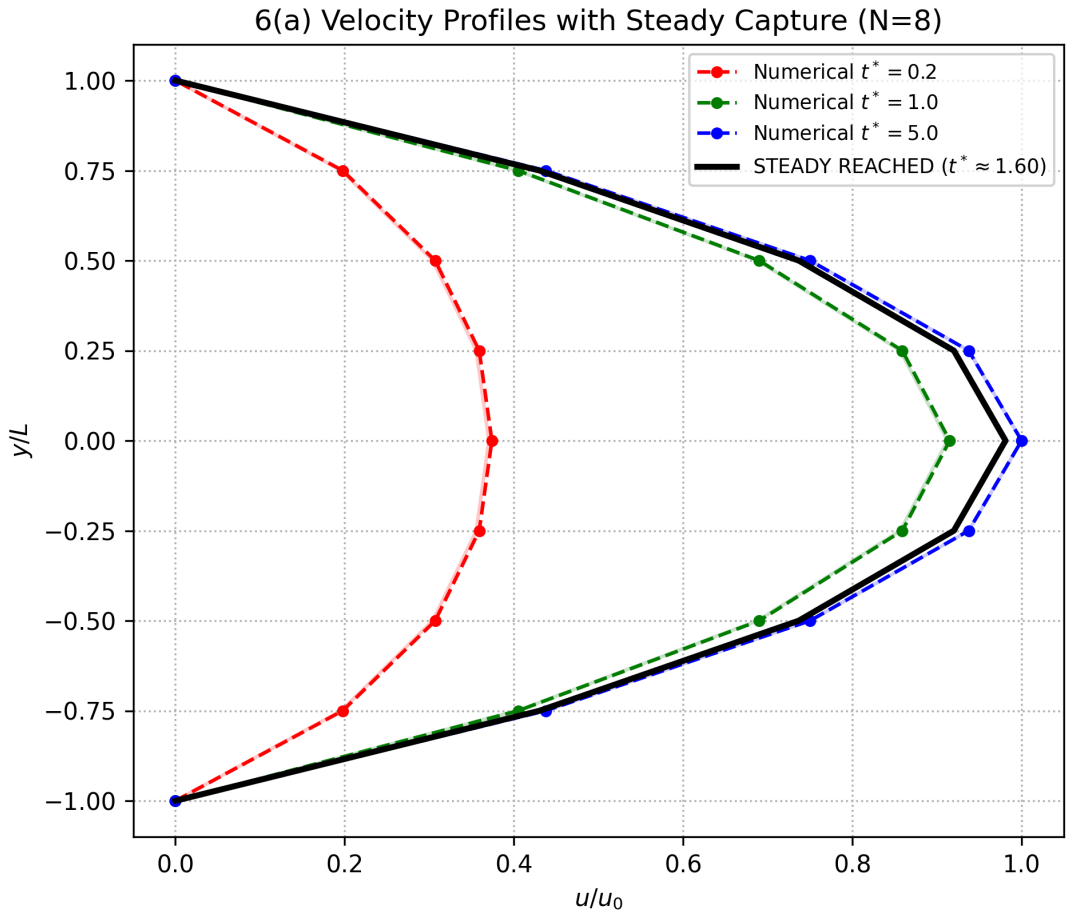
```
Tend = 5.1d0*aL*aL/anu
```

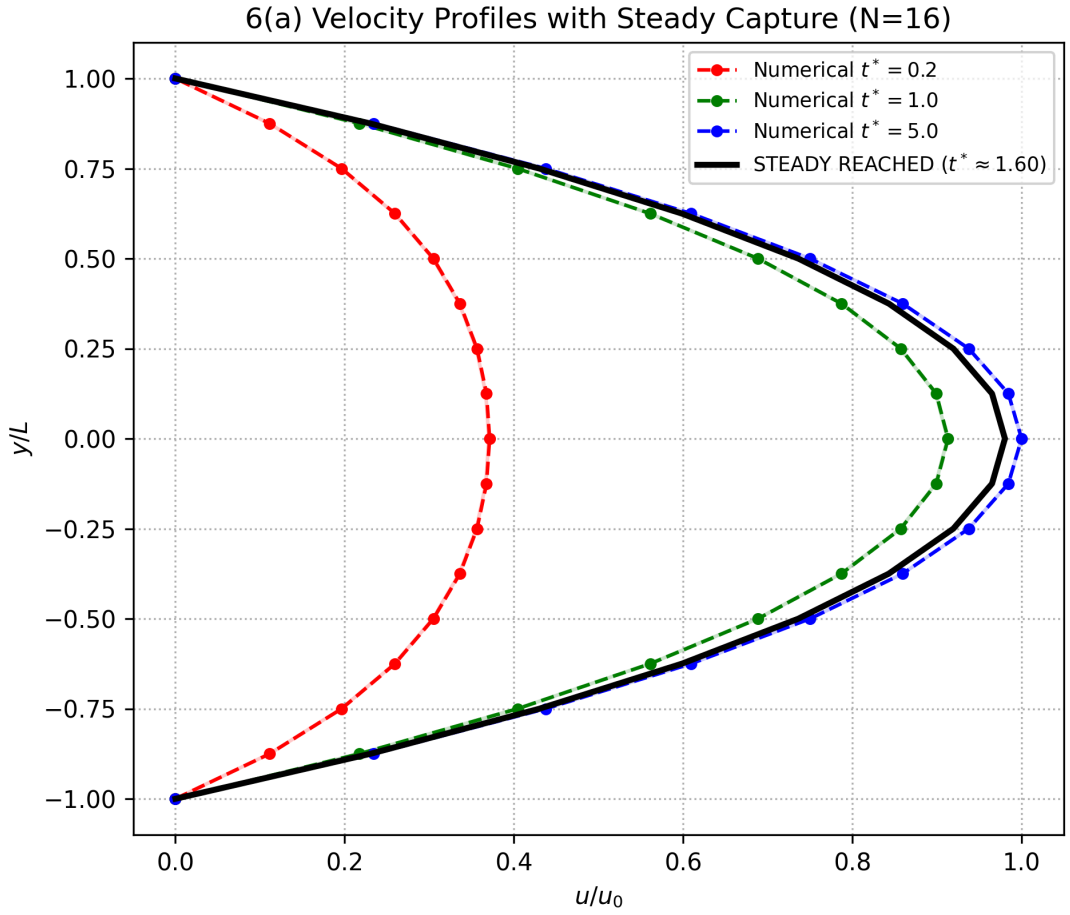
In order to make sure that $\nu t/L^2$ can reach 5.0.

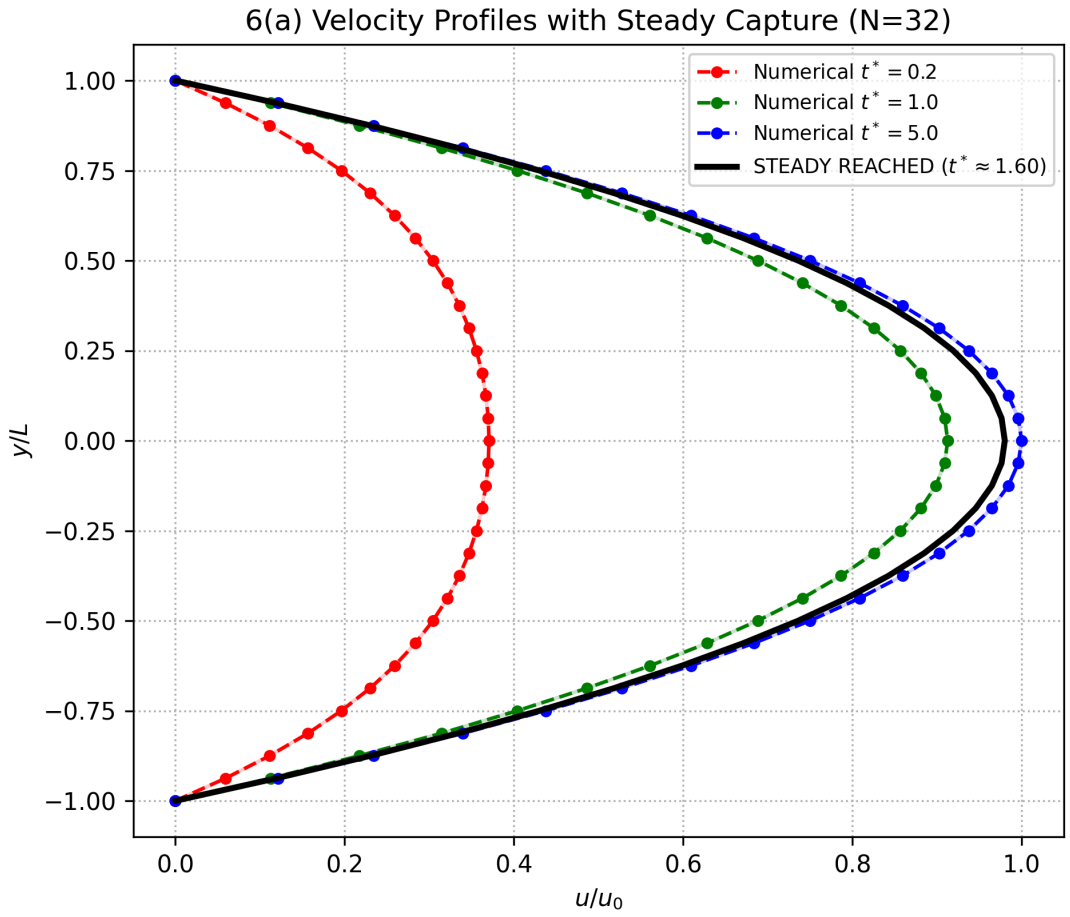
The output is kind like :

```
  1.000000000000  0.125000000000  0.366791952394  0.367657344677  0.002359354610
  1.000000000000  0.250000000000  0.355569160603  0.356405427637  0.002351911040
  1.000000000000  0.375000000000  0.335408172811  0.336189207069  0.002328608309
  1.000000000000  0.500000000000  0.304159140228  0.304851876663  0.002277546005
  1.000000000000  0.625000000000  0.258888585803  0.259455289805  0.002188987979
  1.000000000000  0.750000000000  0.195968921416  0.196372376907  0.002058772830
  1.000000000000  0.875000000000  0.111205424868  0.111415570183  0.001889703810
  1.000000000000  1.000000000000 -0.000000000000  0.000000000000 -1.000000000000
  2.000000000000 -1.000000000000 -0.000000000000  0.000000000000 -1.000000000000
  2.000000000000 -0.875000000000  0.159330017288  0.159550855207  0.001386040886
  2.000000000000 -0.750000000000  0.290294851268  0.290727846850  0.001491571690
  2.000000000000 -0.625000000000  0.395668160098  0.396296351658  0.001587672763
  2.000000000000 -0.500000000000  0.478005656620  0.478804561376  0.001671329083
  2.000000000000 -0.375000000000  0.539546863366  0.540485533478  0.001739737871
  2.000000000000 -0.250000000000  0.582129195088  0.583171490014  0.001790487292
  2.000000000000 -0.125000000000  0.607117621331  0.608223619601  0.001821719928
  2.000000000000  0.000000000000  0.615352519919  0.616480008348  0.001832264259
  2.000000000000  0.125000000000  0.607117621331  0.608223619601  0.001821719928
  2.000000000000  0.250000000000  0.582129195088  0.583171490014  0.001790487292
  2.000000000000  0.375000000000  0.539546863366  0.540485533478  0.001739737871
  2.000000000000  0.500000000000  0.478005656620  0.478804561376  0.001671329083
  2.000000000000  0.625000000000  0.395668160098  0.396296351658  0.001587672763
  2.000000000000  0.750000000000  0.290294851268  0.290727846850  0.001491571690
  2.000000000000  0.875000000000  0.159330017288  0.159550855207  0.001386040886
  2.000000000000  1.000000000000 -0.000000000000  0.000000000000 -1.000000000000
  3.000000000000 -1.000000000000 -0.000000000000  0.000000000000 -1.000000000000
  3.000000000000 -0.875000000000  0.188561951056  0.188763310259  0.001067867627
  3.000000000000 -0.750000000000  0.347634482258  0.348029459320  0.001136184936
```

And the plots are :


6(a) Velocity Profiles with Steady Capture (N=8)
Numerical $t^* = 0.2$
Numerical $t^* = 1.0$
Numerical $t^* = 5.0$
STEADY REACHED ($t^* \approx 1.60$)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y/L
0.0
0.2
0.4
0.6
0.8
1.0
u/u_0


6(a) Velocity Profiles with Steady Capture (N=16)
Numerical $t^* = 0.2$
Numerical $t^* = 1.0$
Numerical $t^* = 5.0$
STEADY REACHED ($t^* \approx 1.60$)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y/L
0.0
0.2
0.4
0.6
0.8
1.0
u/u_0



Acoording to the reports :

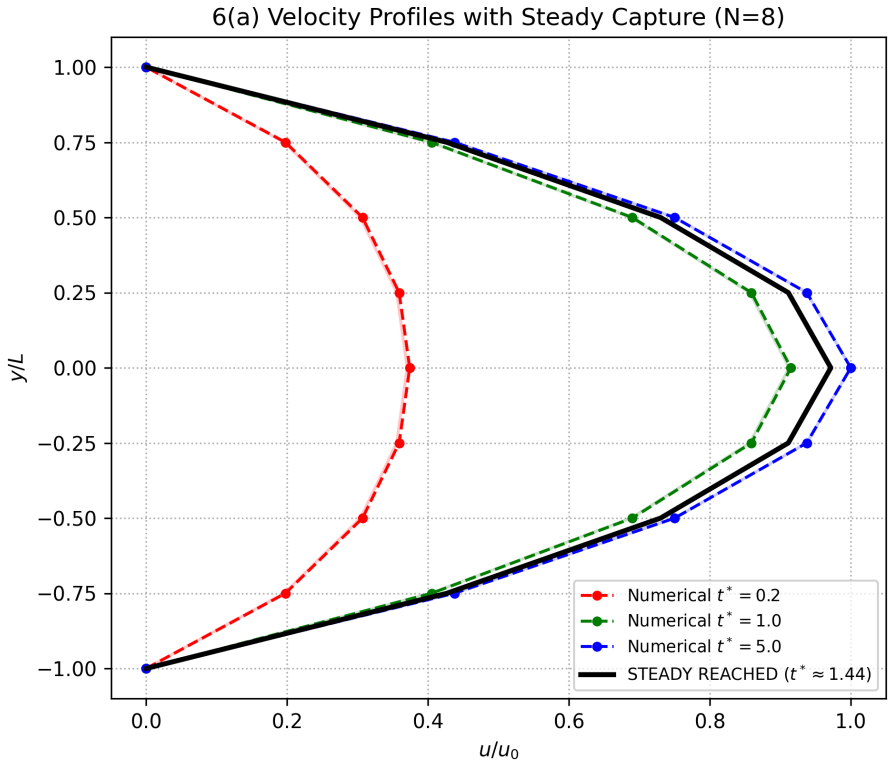
```
N=8: 捕捉到稳态时刻 t* ≈ 1.6000 (Label 8.0)
N=16: 捕捉到稳态时刻 t* ≈ 1.6000 (Label 8.0)
N=32: 捕捉到稳态时刻 t* ≈ 1.6000 (Label 8.0)
```

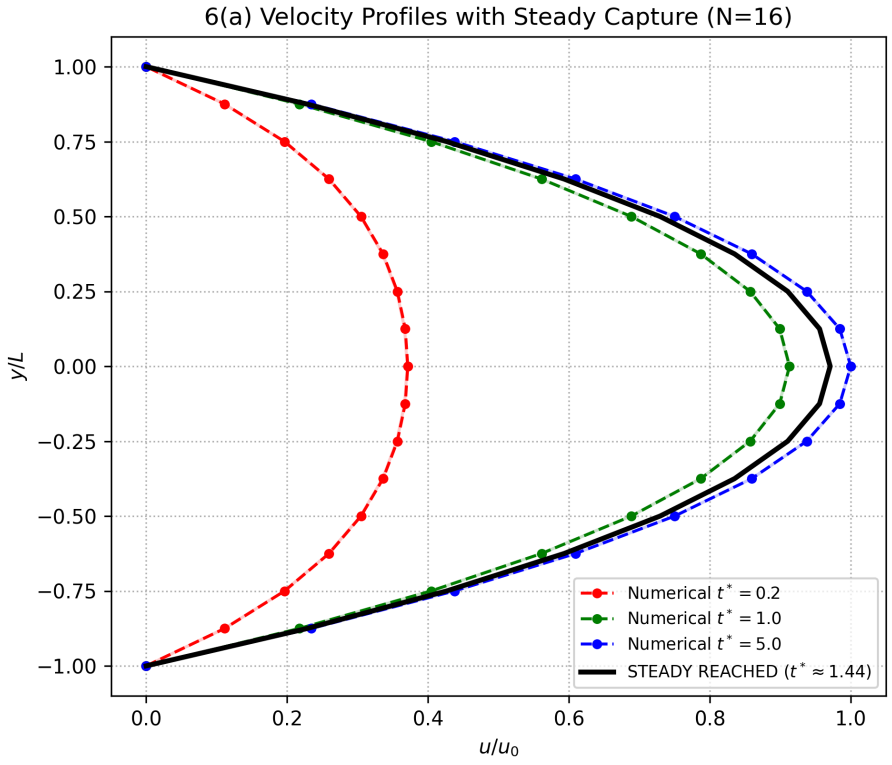
The velocity at the center of the channel (y = 0) reaches the value of 0.97u0 at the dimensionless time is **1.6**, for all the three different grid resolutions. It only shows that the physical solution is between 1.4 ~ 1.8. For all those 3 simulations data output frequencies are the **same**： each t*=0.2.

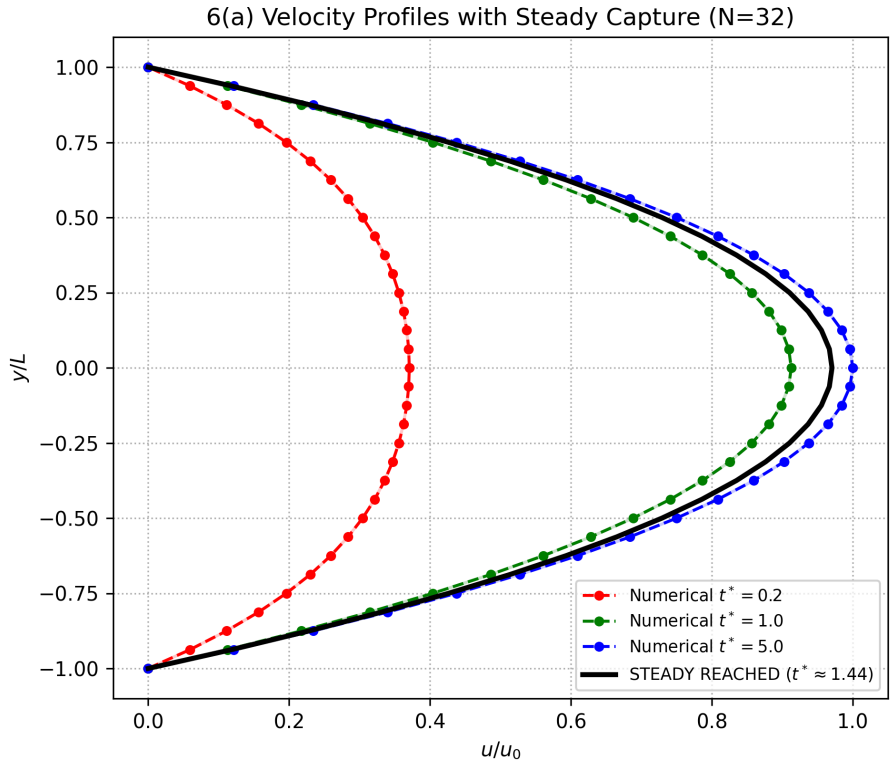
In order the find the differencies between this 3 simulations, I tried to change the **Ntime**, and also repaired the python codes to find the right dimensionless time:

simulation	N	dt	Ntime
1	8	0.2	2
2	16	0.05	8

simulation	N	dt	Ntime
3	32	0.0125	32







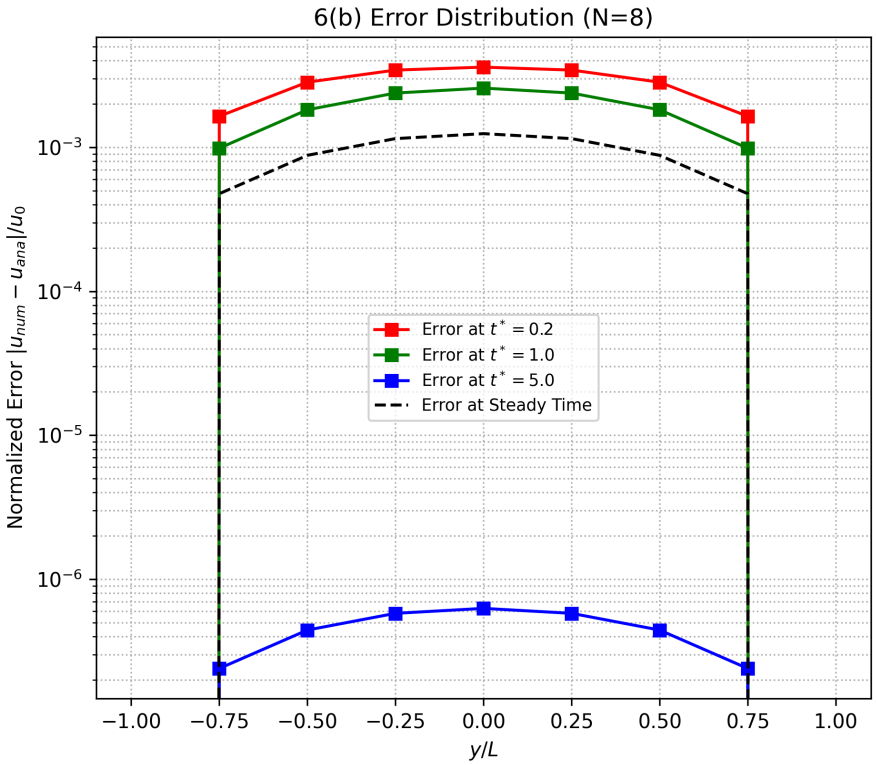
But the results are still **the same** :

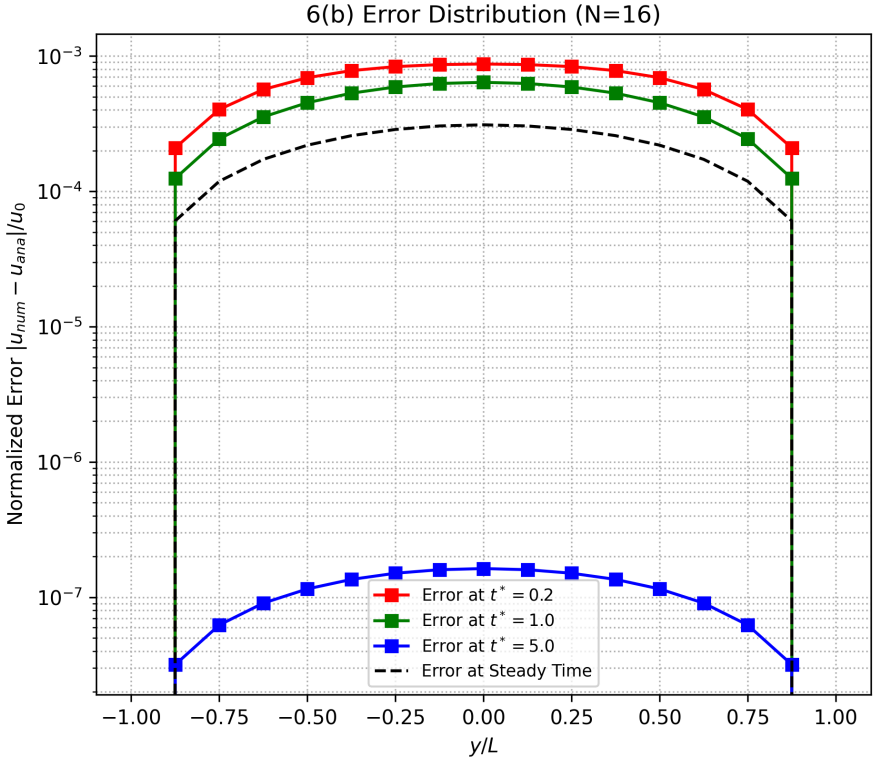
```
N=8: 捕捉到稳态时刻 t* ≈ 1.4400 (Label 36.0)
N=16: 捕捉到稳态时刻 t* ≈ 1.4400 (Label 36.0)
N=32: 捕捉到稳态时刻 t* ≈ 1.4400 (Label 36.0)
```

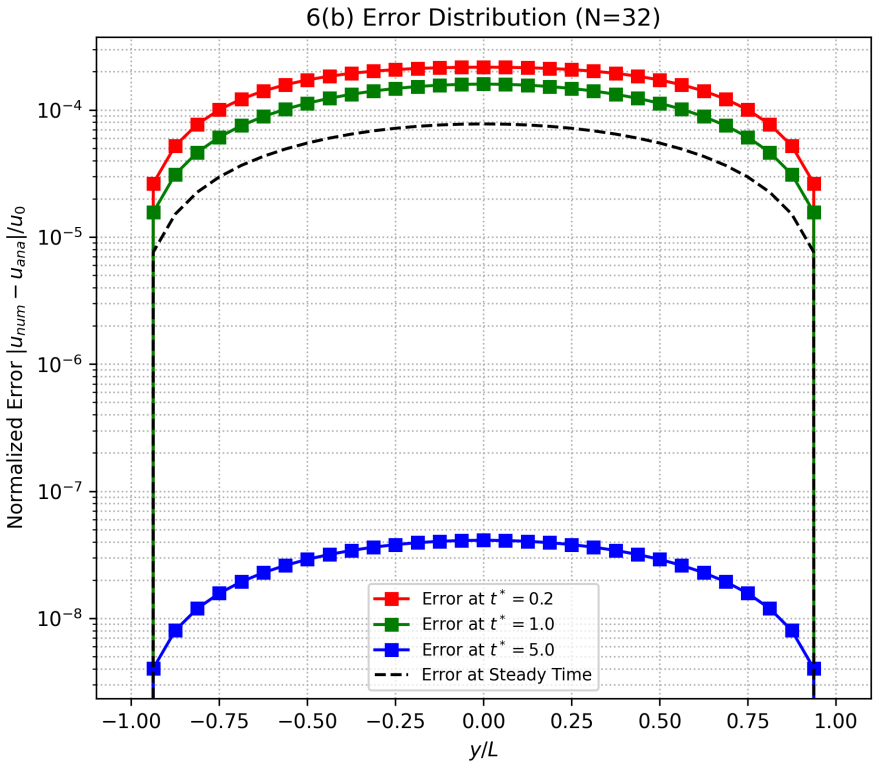
Theoretically, the dimensionless time for steady flow should be the same, for they are the same physical work. But the numerical solution should be a little bit different, maybe it is just a small number error (<0.04)

(b)

The error plots :


6(b) Error Distribution (N=8)
Normalized Error $|u_{num} - u_{ana}|/u_0$
10^{-3}
10^{-4}
10^{-5}
10^{-6}
Error at $t^* = 0.2$
Error at $t^* = 1.0$
Error at $t^* = 5.0$
Error at Steady Time
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
y/L

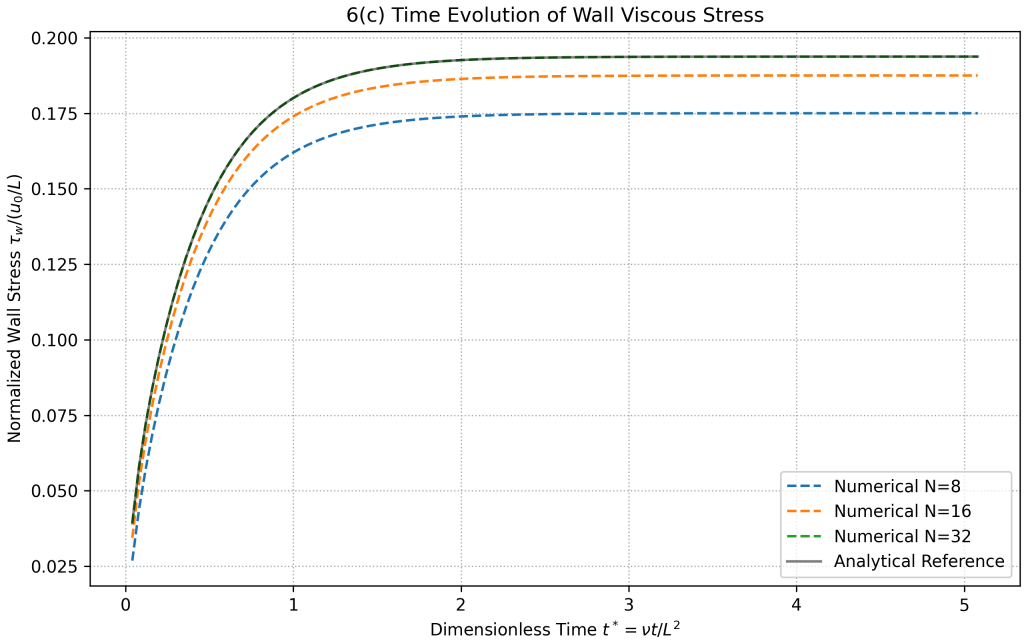

6(b) Error Distribution (N=16)
Normalized Error $|u_{num} - u_{ana}|/u_0$
10^{-3}
10^{-4}
10^{-5}
10^{-6}
10^{-7}
Error at $t^* = 0.2$
Error at $t^* = 1.0$
Error at $t^* = 5.0$
Error at Steady Time
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
y/L



Those plots show that the smaller my grids are, the faster the error converges.

(c)

The wall viscous stress calculated in python(makes my work easier).



The normalized wall viscous stress is (consider $\rho = 1$):

$$\tau_w = \nu \frac{\partial u}{\partial y}|_{wall}$$

As $t \to \infty$, the velocity profile reaches a steady parabolic state:

$$u_{steady}(y) = u_0 \left(1 - \frac{y^2}{L^2}\right)$$

The theoretical velocity gradient at the lower wall ($y = -L$) is:

$$\left.\frac{\partial u}{\partial y}\right|_{y=-L} = u_0 \left(-\frac{2y}{L^2}\right)\Bigg|_{y=-L} = \frac{2u_0}{L}$$

Thus, the normalized wall shear stress τ_w should converge to:

$$\frac{\tau_w}{u_0/L} = \nu \left(\frac{2u_0/L}{u_0/L}\right) = 2\nu$$

Given $\nu = 0.1$, the absolute theoretical steady-state value is **0.2**.

The calculate methods for the numerical and analytical solutions are the same, in each time steps, choose the first interior node u_2 ($u_1 = 0$):

$$\left.\frac{\partial u}{\partial y}\right|_{wall} \approx \frac{u_2 - u_1}{\Delta y} = \frac{u_2}{\Delta y}$$

That is why the analytical solutions are also not approach to the real solution 2.0.

(d)

1. The output method. I am confused whether the code can computer the specific dimensionless time when the flow is steady. Or I need larger **N**.
2. The analytical solutions are also not approach to the real solution 2.0. This is not good, Is there any method to makes this more accurate.

Question 7: Taylor Expansion Error

(It is hard to type so many function in md. So some answer in Q7 and Q8 will be a handwriting version in PDF)

In order to prove : $\frac{d^2 f_i}{dx^2} = \frac{f_{i+1} - 2f_i + f_{i-1}}{\Delta x^2} - \frac{\Delta x^4}{12}\frac{d^4 f_i}{dx^4} + O(\Delta x^4)$

do Taylor Expansion in $x_i + \Delta x$ and $x_i - \Delta x$:

$$f_{i+1} = f_i + \Delta x f_i' + \frac{\Delta x^2}{2!} f_i'' + \frac{\Delta x^3}{3!} f_i''' + \frac{\Delta x^4}{4!} f_i'''' + \frac{\Delta x^5}{5!} f_i^{(5)} + O(\Delta x^6)$$

$$f_{i-1} = f_i - \Delta x f_i' + \frac{\Delta x^2}{2!} f_i'' - \frac{\Delta x^3}{3!} f_i''' + \frac{\Delta x^4}{4!} f_i'''' - \frac{\Delta x^5}{5!} f_i^{(5)} + O(\Delta x^6)$$

∴ added the two eq. above :

$$f_{i+1} - 2f_i + f_{i-1} = 2\left(\frac{\Delta x^2}{2!} f_i^{(2)} + \frac{\Delta x^4}{4!} f_i^{(4)} + O(\Delta x^6) \cdots\right)$$

$$\Rightarrow \Delta x^2 f_i'' = f_{i+1} - 2f_i + f_{i-1} - \frac{\Delta x^4}{12} f_i^{(4)} + O(\Delta x^6)$$

$$\Rightarrow \frac{d^2 f(x_i)}{dx^2} = \frac{f_{i+1} - 2f_i + f_{i-1}}{\Delta x^2} - \frac{\Delta x^2}{12}\frac{d^4 f(x_i)}{dx^4} + O(\Delta x^4)$$

$f(x) = \sin(\pi x/2) + 0.1x^5$

$f'(x) = \frac{\pi}{2}\cos(\pi x/2) + 0.5x^4$

$f''(x) = -\frac{\pi^2}{4}\sin(\pi x/2) + 2x^3$

$f''_{ext}(1) = -\frac{\pi^2}{4} + 2 \approx -0.467401$

In numerical method :

$f(1.1) = \sin(\frac{1.1}{2}\pi) + 0.1(1.1)^5 \approx 1.148739$

$f(0.9) = \sin(\frac{0.9\pi}{2}) + 0.1(0.9)^5 \approx 1.046737$

$f(1) = \sin(\frac{\pi}{2}) + 1 = 1.1$

$f''_{num}(1) = \frac{f(1.1) - 2f(1) + f(0.9)}{0.1^2} = -0.4524$

The actual error : $E_{act} = \frac{|f''_{num}(1) - f''_{ext}(1)|}{|f''_{ext}(1)|} = 3.2\%$ → ≈ 0.015001

The Leading-order Estimate: $E_L = \left(\frac{(0.1)^2}{12} f^{(4)}(1) \Big/ |f''_{ext}(1)|\right) = 3.2\%$ → ≈ 0.013073

Question 8: time-dependent differential equation

(a) Using Taylor expansion, determine α, β, and γ, such that the scheme has the least truncation error.

Q.8.(a) do Taylor Expasion at $t^{(n)}$, say $F^{(n)} = F(u^{(n)}, t^{(n)})$

$$u^{(n+1)} = u^{(n)} + \Delta t\, u'^{(n)} + \frac{\Delta t^2}{2!} u''^{(n)} + \frac{\Delta t^3}{3!} u'''^{(n)} + O(\Delta t^4)$$

$$\because F^{(n)} = u'^{(n)}$$

$$\therefore \frac{u^{(n+1)} - u^{(n)}}{\Delta t} = F^{(n)} + \frac{\Delta t}{2} F'^{(n)} + \frac{\Delta t^3}{6} F''^{(n)} + O(\Delta t^3) = LHS$$

And for $F^{(n-1)}, F^{(n-2)}$:

$$F^{(n-1)} = F^{(n)} - \Delta t F'^{(n)} + \frac{\Delta t^2}{2} F''^{(n)} + O(\Delta t^3)$$

$$F^{(n-2)} = F^{(n)} - 2\Delta t F'^{(n)} + \frac{(2\Delta t)^2}{2} F''^{(n)} + O(\Delta t^3)$$

$$\therefore RHS = F^{(n)}(\alpha+\beta+\gamma) + \Delta t F'^{(n)}(-\beta-2\gamma) + \Delta t^2 F''^{(n)}\left(\frac{\beta}{2}+2\gamma\right)$$

$$\because LHS = RHS:$$

$$\begin{cases} \alpha+\beta+\gamma = 1 \\ -\beta-2\gamma = \frac{1}{2} \\ \frac{\beta}{2}+2\gamma = \frac{1}{6} \end{cases} \Rightarrow \begin{cases} \alpha = \frac{23}{12} \\ \beta = -\frac{4}{3} \\ \gamma = \frac{5}{12} \end{cases}$$

(b) What is the order of accuracy of the resulting scheme?

Of course is :

$$\mathcal{O}(dt^3)$$

EOF